



Echara — Database Diagram & API Endpoint Maps (MVP)

Version: 1.0

Scope: Buyer, Vendor, Admin web (UK-only)

Note: Mirrors the previously agreed MVP spec. AI recommendations and Stripe payments are Phase 2.

1) High-Level Entity–Relationship (ER) Diagram

```
classDiagram
    direction LR

    class Users {
        +ObjectId _id
        +enum role (buyer|vendor|admin)
        +string name
        +string email
        +string avatarUrl
        +ObjectId[] favourites (Listing)
        +date createdAt
        +date updatedAt
        +date gdprDeletedAt?
    }

    class Vendors {
        +ObjectId _id
        +ObjectId userId (Users)
        +string businessName
        +number yearsInBusiness
        +string description
        +string[] tags
        +object socialLinks{instagram?, tiktok?, website?}
        +string address
        +object geo{lat,lng}
        +string heroImage
        +string[] gallery
        +enum status(pending|approved|suspended)
        +enum subscriptionStatus(trial|active|past_due|canceled)
        +date trialEndDate
        +number adSpendMonthly?
        +date createdAt
        +date updatedAt
    }

    class Listings {
        +ObjectId _id
```

```

+ObjectId vendorId (Vendors)
+string category
+object priceRange{min,max,currency}
+string description
+string[] gallery
+object location{lat,lng,address}
+object[] availability[{startISO,endISO,source}]
+object policies{depositPercent?, cancellationWindowDays?, refundRules?}
+object ratingSummary{avgOverall,count,criteriaAvg}
+bool isFeatured
+bool isVisible
+date createdAt
+date updatedAt
}

class Bookings {
+ObjectId _id
+ObjectId buyerId (Users)
+ObjectId vendorId (Vendors)
+ObjectId listingId (Listings)
+enum status(pending|confirmed|declined|completed|canceled)
+object requestedSlot{startISO,endISO}
+date approvedAt?
+date declinedAt?
+date completedAt?
+date canceledAt?
+string cancelReason?
+object paymentLogged{amountGBP,method,dateISO,notes??}
+ObjectId messagesThreadId (Chats)
+date createdAt
+date updatedAt
}

class Chats {
+ObjectId _id
+ObjectId buyerId (Users)
+ObjectId vendorId (Vendors)
+ObjectId listingId (Listings)
+Message[] messages
+date lastMessageAt
}

class Message {
+ObjectId _id
+ObjectId senderId (Users)
+enum type(text|image|file)
+string contentUrlOrText
+date createdAt
+ObjectId[] seenBy
}

```

```

class Reviews {
  +ObjectId _id
  +ObjectId bookingId (Bookings)
  +ObjectId buyerId (Users)
  +ObjectId vendorId (Vendors)
  +ObjectId listingId (Listings)
  +object ratings{quality,communication,value,professionalism}
  +number overall
  +string comment
  +string vendorReply?
  +date repliedAt?
  +bool hiddenByAdmin?
  +date createdAt
  +date updatedAt
}

```

```

class Notifications {
  +ObjectId _id
  +ObjectId userId (Users)
  +string type
  +string title
  +string body
  +object data
  +bool isRead
  +date createdAt
}

```

```

class AdminLogs {
  +ObjectId _id
  +ObjectId performedByAdminId (Users)
  +string action
  +string targetType
  +ObjectId targetId
  +object metadata
  +date createdAt
}

```

```

Users "1" <--> "0..1" Vendors : owns
Vendors "1" <--> "0..*" Listings : publishes
Users "1" <--> "0..*" Bookings : makes
Vendors "1" <--> "0..*" Bookings : receives
Listings "1" <--> "0..*" Bookings : for
Users "1" <--> "0..*" Chats : participates
Vendors "1" <--> "0..*" Chats : participates
Listings "1" <--> "0..*" Chats : context
Bookings "1" <--> "0..1" Chats : thread
Bookings "1" <--> "0..1" Reviews : generates
Users "1" <--> "0..*" Reviews : writes
Vendors "1" <--> "0..*" Reviews : receives

```

2) Key Indexes & Query Notes

- **Listings**

- category: 1, location.lat: 1, location.lng: 1 (or 2dsphere index on location for geo queries)

- Text index on description and tags

- **Bookings**

- vendorId: 1, status: 1, requestedSlot.startISO: 1
- Unique guard against overlapping confirmed slots per vendor/listing

- **Chats**

- buyerId: 1, vendorId: 1, lastMessageAt: -1

- **Reviews**

- vendorId: 1, createdAt: -1

- **AdminLogs**

- createdAt: -1, action: 1, targetType: 1

3) API Surface (REST) — Endpoint Map

Base path: /api/v1

```
flowchart TB
    subgraph Auth
        A1[/POST auth/register/]
        A2[/POST auth/login/]
        A3[/POST auth/oauth/callback/]
        A4[/GET me/]
        A5[/DELETE me/]
        A6[/GET me/favourites/]
        A7[/POST me/favourites/:listingId/]
    end

    subgraph Vendors
        V1[/POST vendors/]
        V2[/GET vendors/:id/]
        V3[/GET vendors/:id/insights/]
        V4[/POST vendors/:id/calendar/google/connect/]
        V5[/DELETE vendors/:id/calendar/google/disconnect/]
    end

    subgraph Listings
        L1[/POST listings/]
        L2[/PUT listings/:id/]
        L3[/DELETE listings/:id/]
        L4[/GET listings/:id/]
        L5[/GET listings/search/]
    end

    subgraph Bookings
```

```

    B1[/POST bookings/]
    B2[/GET bookings/:id/]
    B3[/POST bookings/:id/approve/]
    B4[/POST bookings/:id/decline/]
    B5[/POST bookings/:id/cancel/]
    B6[/POST bookings/:id/complete/]
    B7[/POST bookings/:id/payment-log/]
end

subgraph Chat
    C1[/GET chats/]
    C2[/GET chats/:id/]
    C3{{WS chat:join}}
    C4{{WS chat:message}}
    C5{{WS chat:typing}}
    C6{{WS chat:seen}}
end

subgraph Reviews
    R1[/POST reviews/]
    R2[/GET vendors/:id/reviews/]
    R3[/POST reviews/:id/reply/]
    R4[/DELETE reviews/:id/]
end

subgraph Notifications
    N1[/GET notifications/]
    N2[/POST notifications/:id/read/]
end

subgraph Admin
    AD1[/GET admin/vendors/]
    AD2[/POST admin/vendors/:id/status/]
    AD3[/POST admin/listings/:id/visibility/]
    AD4[/POST admin/reviews/:id/hide/]
    AD5[/GET admin/analytics/overview/]
    AD6[/GET admin/logs/]
end

A4-->V2
A4-->L5
L5-->L4
L4-->B1
B1-->C1
C1-->C3
B1-->B2
B2-->R1
V1-->L1
AD1-->AD2
AD3-->L4
AD4-->R2

```

4) Sequence Diagrams (Key Flows)

4.1 Booking Request (No In-App Payment)

```
sequenceDiagram
    autonumber
    participant Buyer
    participant WebAPI as Echara API
    participant Vendor
    participant Notif as Notifications
    participant DB as MongoDB

    Buyer->>WebAPI: GET /listings/search (filters)
    WebAPI->>DB: Query listings (indexes)
    DB-->>WebAPI: Results
    WebAPI-->>Buyer: Listings

    Buyer->>WebAPI: POST /bookings (listingId, slot)
    WebAPI->>DB: Create booking{status: pending}
    DB-->>WebAPI: _id
    WebAPI->>Notif: Push + Email to Vendor (pending)
    Notif-->>Vendor: New booking request

    Vendor->>WebAPI: POST /bookings/:id/approve
    WebAPI->>DB: Update status->confirmed + approvedAt
    DB-->>WebAPI: OK
    WebAPI->>Notif: Push to Buyer (confirmed)
    Notif-->>Buyer: Booking confirmed

    Note over Vendor,WebAPI: Vendor may later POST /bookings/:id/payment-log
```

4.2 Realtime Chat Setup

```
sequenceDiagram
    autonumber
    participant Client as Buyer/Vendor App
    participant Socket as Socket.IO Server
    participant API as Echara API
    participant DB as MongoDB

    Client->>API: GET /chats (list)
    API->>DB: Find chats by user
    DB-->>API: Chat list
    API-->>Client: Chat summaries

    Client->>Socket: chat:join {chatId}
    Socket-->>Client: ack joined
```

```
Client->>Socket: chat:message {chatId, type, payload}
Socket->>DB: Persist message
Socket-->>Client: echo (sender)
Socket-->>Others: broadcast to other participant
```

5) Error Model & HTTP Status Conventions

- **2xx** success; **400** validation; **401** unauthenticated; **403** unauthorized; **404** not found; **409** conflict (e.g., overlapping booking); **429** rate-limited; **500** server error.
 - All error payloads return `{ code, message, details? }`.
-

6) Swagger / OpenAPI Stub (for Devs to Extend)

```
openapi: 3.0.3
info:
  title: Echara API
  version: 1.0.0
servers:
  - url: /api/v1
paths:
  /listings/search:
    get:
      summary: Search listings
      parameters:
        - in: query
          name: category
          schema: { type: string }
        - in: query
          name: lat
          schema: { type: number }
        - in: query
          name: lng
          schema: { type: number }
        - in: query
          name: radius
          schema: { type: number, default: 25 }
        - in: query
          name: minPrice
          schema: { type: number }
        - in: query
          name: maxPrice
          schema: { type: number }
        - in: query
          name: rating
          schema: { type: number, minimum: 1, maximum: 5 }
        - in: query
          name: page
```

```

        schema: { type: integer, default: 1 }
      - in: query
        name: limit
        schema: { type: integer, default: 20 }
    responses:
      '200':
        description: OK
/bookings:
  post:
    summary: Create booking request
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [listingId, requestedSlot]
            properties:
              listingId: { type: string }
              requestedSlot:
                type: object
                required: [startISO, endISO]
                properties:
                  startISO: { type: string, format: date-time }
                  endISO: { type: string, format: date-time }
    responses:
      '201': { description: Created }
      '400': { description: Bad Request }
      '401': { description: Unauthorized }

```

7) Deployment Topology (MVP)

```

graph LR
  subgraph Clients
    A[React Native Apps]
    B[Next.js Web]
    C[Admin Web]
  end

  subgraph Vercel
    API[Serverless API Routes]
    WEB[Static/SSR Web]
  end

  subgraph Realtime Host
    SIO[Node + Socket.IO]
  end

```



```

subgraph Cloud Services
  MDB[(MongoDB Atlas)]
  FCM[Firebase Cloud Messaging]
  FAuth[Firebase Auth]
  FStore[Firebase Storage]
  GMaps[Google Maps]
  GCal[Google Calendar]
  Email[Resend/SendGrid]
end

A-->WEB
B-->WEB
C-->WEB
WEB-->API
WEB-->SIO
API---MDB
SIO---MDB
API---FAuth
API---FStore
API---GMaps
API---GCal
API---Email
WEB---FCM

```

8) Post-MVP Placeholders

- **AI Recommendations:** new service `recommendation` reading anonymised events; endpoints `/recommendations/:userId`.
- **Stripe Connect:** payments service; webhooks `/webhooks/stripe`.
- **CSV Exports:** vendor dashboard → export endpoints with signed URLs.

9) Appendix — Reference Category Seeds (Editable)

Venues, Photographers, Videographers, Caterers, Cake Makers, Florists, DJs, Live Bands, Decor & Styling, Car Hire, Hair & Makeup, Suits, Dresses, Invitations, Lighting & AV, Marquees & Tents, Event Planners, Officiants, Entertainment, Photo Booths, Favours & Gifts, Security, Clean-up

Implementation tip: Keep a `FeatureFlags` collection to toggle admin-only features during staging. Ensure all diagrams and endpoint lists here are kept in sync with the codebase's autogenerated OpenAPI document.